

Simulating Digitised Input Textures in Riley

1 Purpose

Riley normally accepts floating-point texture samples, reconstructs the texture continuously, performs the pixel or point-spread-function integral, and only then applies the output scaling, clamping, and final camera digitisation. This ordering is retained when simulating a digitised input texture.

The input texture is still stored and passed to Riley as floating-point data. Input digitisation is simulated only by rounding each floating-point texel to a prescribed grey-level increment before rendering. The post-integration scale, clamp, and final image quantisation remain unchanged.

This creates a controlled input representation error while preserving Riley’s existing floating-point texture-to-image pipeline.

2 Reference additive texture

Let the sampled additive texture be

$$T[i, j] \in \mathbb{R}, \quad (1)$$

where overlapping Gaussian blobs or disks may give

$$T[i, j] > 1. \quad (2)$$

No pre-render clamping is applied. Values greater than unity remain valid because the additive field is scaled and clamped only after rendering and pixel integration.

3 Simulated input digitisation

For a nominal input precision of b bits over the normalised interval $[0, 1]$, define the quantisation increment

$$\Delta_b = \frac{1}{2^b - 1}. \quad (3)$$

The digitised texture samples are

$$T_b[i, j] = \Delta_b \operatorname{round}\left(\frac{T[i, j]}{\Delta_b}\right) = \frac{\operatorname{round}[(2^b - 1)T[i, j]]}{2^b - 1}. \quad (4)$$

Equation (4) changes only the precision of the floating-point texels. It does not restrict their range. For example, with $b = 8$,

$$\Delta_8 = \frac{1}{255}, \quad (5)$$

and a texel value greater than one is rounded to the nearest multiple of $1/255$ while remaining greater than one.

The phrase “ b -bit-equivalent input precision” is therefore preferable to “ b -bit texture”, because the decoded texture values are not restricted to the integer code range $0, \dots, 2^b - 1$.

4 Rendering pipeline

For the original floating-point texture, the Riley pipeline is

$$T \xrightarrow{\mathcal{I}} \tilde{T}(\mathbf{x}) \xrightarrow{\mathcal{P}_h} P_h \xrightarrow{\mathcal{C}} G_h \xrightarrow{Q_c} I_h, \quad (6)$$

where

- $\mathcal{I}$ is texture reconstruction;
- $\mathcal{P}_h$ is the finite SSAA or pixel-integration operator;
- $\mathcal{C}$ is the existing post-integration scaling and clamping operation;
- Q_c is the final camera digitisation at output bit depth c .

For the simulated digitised input texture, the only added operation is the input rounding:

$$T \xrightarrow{Q_b^{\text{in}}} T_b \xrightarrow{\mathcal{I}} \tilde{T}_b(\mathbf{x}) \xrightarrow{\mathcal{P}_h} P_{h,b} \xrightarrow{\mathcal{C}} G_{h,b} \xrightarrow{Q_c} I_{h,b}. \quad (7)$$

The same reconstruction, SSAA, scale, clamp, and output digitisation settings must be used for T and T_b .

5 Why pre-render clamping must not be added

The additive texture should not be clipped to $[0, 1]$ before rendering. In general,

$$\mathcal{P}[\min(T, 1)] \neq \min(\mathcal{P}[T], 1). \quad (8)$$

Pre-render clamping therefore changes the image-formation model. It would remove additive overlap before the pixel integral rather than applying the intended saturation rule after integration.

The correct simulated input digitisation changes only the texel increment. It leaves the additive range and the placement of scale and clamp operations unchanged.

6 Demonstrating that SSAA cannot remove input digitisation error

Choose a fixed spatial texture resolution and a fixed texture realisation. Render both the original and rounded textures over a sequence of increasing SSAA levels:

$$I_h = \mathcal{R}_h[T], \quad (9)$$

$$I_{h,b} = \mathcal{R}_h[T_b]. \quad (10)$$

At sufficiently high SSAA,

$$I_h \rightarrow I_\infty, \quad (11)$$

$$I_{h,b} \rightarrow I_{\infty,b}. \quad (12)$$

Because $T_b \neq T$ at fixed b , the limiting images will generally differ:

$$I_{\infty,b} \neq I_\infty. \quad (13)$$

Thus increasing SSAA removes pixel-quadrature error but cannot remove input digitisation error.

A clean demonstration uses two metrics.

Self-convergence of the rounded texture

$$e_{\text{self}}(h, b) = \|I_{h,b} - I_{\text{ref},b}\|, \quad (14)$$

where $I_{\text{ref},b}$ is produced with the highest available SSAA using the same rounded input texture. This error should tend to zero.

Error relative to the floating-point texture

$$e_{\text{input}}(h, b) = \|I_{h,b} - I_{\text{ref}}\|, \quad (15)$$

where I_{ref} is the high-SSAA result from the original floating-point texture. This error should approach a non-zero plateau set by the input texture quantisation.

The expected behaviour is therefore

$$e_{\text{self}} \rightarrow 0, \quad e_{\text{input}} \rightarrow e_{\text{floor}} > 0. \quad (16)$$

This directly shows that SSAA can accurately integrate the reconstructed rounded texture, but cannot restore grey-level information discarded before rendering.

7 Recommended minimal experiment

A compact study can use a single additive texture realisation and a single sufficiently high spatial texture oversampling level. For each selected input precision b :

1. Generate the original $f64$ additive texture T .
2. Form T_b using Equation (4), without clamping.
3. Render T and T_b with identical texture reconstruction, SSAA, scaling, clamping, and output bit depth.
4. Increase only SSAA.
5. Plot self-convergence and error relative to the floating-point reference.

Suitable image metrics are root-mean-square error and maximum absolute error. If required, the same comparison may be propagated through DIC, but this is not necessary to demonstrate the irreducible image-level floor.

8 Interpretation

The experiment does not reproduce every feature of a real experimental image, which would also contain optical blur, sensor integration, noise, clipping, and camera-specific processing. It isolates one narrower mechanism: finite grey-level precision in the texture supplied to Riley.

The result should be described as follows:

Input digitisation was simulated by rounding floating-point additive texels to the normalised increment associated with a nominal b -bit representation, while retaining values greater than unity. Riley’s existing post-integration scaling, clipping, and final camera digitisation were left unchanged. Increasing SSAA converged the rendering of the rounded texture but could not remove the non-zero error relative to the original floating-point texture, because the discarded input grey-level information is not recoverable through finer pixel quadrature.